Voice-Based AI Receptionist Agent

for Smart College Communication

Using RAG and Agentic AI

Combined Project Proposal — Phase 1 Implemented, Phase 2 Proposed | Mentor Review Document

Rajalakshmi Engineering College | B.E. Computer Science and Engineering

---

Abstract

Educational institutions receive numerous telephone inquiries every day regarding admissions, courses, departments, fee structure, placements, hostel facilities, academic schedules, and student services. Handling every call manually increases the workload of reception staff, leads to longer waiting times, and may result in inconsistent responses. To address these challenges, this project proposes a Voice-Based AI Receptionist Agent capable of answering calls autonomously using speech recognition, Retrieval-Augmented Generation (RAG), Large Language Models (LLMs), and Agentic AI.

In Phase 1 — which has been implemented and evaluated — the voice agent operates on publicly available college information covering admissions, departments, faculty details, campus facilities, placements, events, and academic regulations. Incoming calls are handled by the AI receptionist via a SIP-based telephony stack (Asterisk), which captures caller audio, converts speech to text using Faster-Whisper, retrieves relevant institutional knowledge from a ChromaDB vector store populated with 767 indexed chunks, generates a grounded response using a locally hosted Qwen3:4B language model via Ollama, and delivers the synthesised response back to the caller using Piper TTS. The agent is designed to answer only from the institutional knowledge base and to decline or seek clarification when the retrieved context is insufficient to support a confident response.

Phase 2 — planned as a future extension — will introduce an authenticated Agentic AI layer that integrates with the college ERP or DigiCampus platform to answer personalised student queries such as attendance, internal marks, fee status, examination results, and placement eligibility. It will further implement intelligent call escalation to a human receptionist when the query requires administrative judgment or falls outside the agent's defined scope.

Phase 1 was evaluated on a set of 20 test queries, comprising 18 normal institutional information queries and 2 negative (out-of-scope) queries. The evaluation recorded a Hit@5 retrieval accuracy of 100%, an answer accuracy of 100%, and a refusal accuracy of 100% on out-of-scope queries. A separate latency evaluation measured an average end-to-end AI processing time of 21.35 seconds, comprising 0.25 seconds for speech-to-text, 0.07 seconds for RAG retrieval, 18.55 seconds for LLM generation, and 2.46 seconds for text-to-speech synthesis. LLM generation is the dominant contributor to latency and is identified as the primary bottleneck for future optimisation.

The proposed solution aims to reduce receptionist workload, provide consistent and knowledge-grounded responses, improve caller experience, and is designed to support continuous availability for routine inquiries when deployed as a continuously running service, while establishing a modular and evaluable AI pipeline suitable for extension toward personalised student services.

1. Background and Motivation

College reception desks are a critical point of contact for prospective students, parents, and current students, yet they operate under significant resource constraints. A single receptionist handles a mix of walk-in visitors, email inquiries, and telephone calls simultaneously, often under pressure during admissions season, examination periods, and placement drives. The result is extended hold times, inconsistent information delivery depending on which staff member answers, and complete unavailability outside office hours.

The Rajalakshmi Engineering College (REC) website already includes an entry point for an AI assistant (the “Ask Aadhi” widget on the homepage), demonstrating institutional readiness for AI-assisted interaction. This project extends that direction by targeting the telephone communication channel, replacing or augmenting it with an intelligent voice agent that handles routine institutional queries autonomously. Phase 1 establishes the foundational voice-and-retrieval pipeline over a locally hosted architecture; Phase 2 will extend it with authenticated, personalised student services and intelligent call escalation.

2. Problem Statement

The college currently has no automated voice system capable of understanding natural-language telephone queries and providing accurate, context-grounded responses drawn from institutional data. Specific gaps addressed by this project are:

- High call volume to reception staff for routine, repeatable queries (fee structure, admission criteria, hostel availability, department contacts) that do not require human judgment.
- No out-of-hours coverage: calls outside office hours go unanswered.
- Inconsistency: different staff members may provide different answers to the same question depending on individual knowledge and access to current documents.
- No integration between telephone inquiries and institutional ERP data for personalised student queries (planned Phase 2).

> *Phase 1 (Implemented): Build a locally hosted voice AI that autonomously handles telephone inquiries about publicly available institutional information using Asterisk/SIP telephony, Faster-Whisper STT, a ChromaDB-backed RAG pipeline, a locally hosted Qwen3:4B LLM via Ollama, and Piper TTS — with no access to personal student data.*
>
> *Phase 2 (Planned): Extend the agent with authenticated agentic tool calling to retrieve personalised student data from DigiCampus via API, and implement intelligent escalation to a human receptionist for out-of-scope queries.*

3. Proposed System Overview

3.1 Phase 1 — Implemented Call Flow

The following sequence describes how an inbound call is handled by the Phase 1 AI receptionist:

1. The caller dials the configured SIP extension. Asterisk 22.10.1 receives the call from a MicroSIP softphone and routes it through an AudioSocket channel to the Python audio gateway.
2. The Python audio gateway receives raw PCM audio from Asterisk via the AudioSocket protocol. WebRTC VAD (Voice Activity Detection) segments the audio stream to detect speech.
3. Detected speech segments are transcribed to text by Faster-Whisper (small model, CUDA-accelerated, float16 inference).
4. The transcribed query is passed to the RAG service, which embeds the query using Sentence Transformers, retrieves the most relevant chunks from ChromaDB, and applies relevance-based reranking. Retrieved chunks carry source and page metadata.
5. The retrieved context, together with the caller's query, is sent to the locally hosted Qwen3:4B model via Ollama, which generates a grounded natural-language response. If no sufficiently relevant context is found, the agent is instructed to decline politely rather than speculate.
6. Piper TTS (en_US-lessac-medium) synthesises the text response into speech. The resulting 8 kHz PCM audio is returned to Asterisk via AudioSocket and played back to the caller.

3.2 Phase 2 — Planned Extensions (Not Yet Implemented)

Phase 2 is planned as a future development stage and is not part of the current implementation. Conceptually, Phase 2 extends the Phase 1 voice pipeline with a query-classification and agentic-decision layer that routes each caller query to public RAG knowledge, authenticated DigiCampus data, or human escalation:

Caller — Phase 1 Voice Pipeline

↓ Query Classification / Agentic Decision (Planned)

↓ Public Query → RAG Knowledge Base | Personal Query → Authentication → DigiCampus/API → Personalised Data

↓ Voice Response, or → Human Receptionist Escalation with Conversation Summary (when judgment, approval, confidential handling, unsupported operation, failed authentication, or low confidence is involved)

The following capabilities are targeted as part of this planned extension:

- Authenticated agentic queries: After verifying caller identity (Student ID and PIN or OTP), the agent will invoke DigiCampus API tools to retrieve and speak back personalised student data such as attendance, examination results, fee status, and placement eligibility.
- Intelligent escalation: When a query cannot be resolved by the AI (requires administrative judgment, involves data outside the system's scope, or falls below a confidence threshold), the agent will announce the transfer and route the call to the available human receptionist, forwarding a transcript summary of the conversation.

- Agentic tool-calling layer: The LLM will be equipped with tool-use capabilities to decide dynamically whether to answer from the RAG knowledge base, invoke a DigiCampus data tool, or escalate the call.

4. System Architecture

4.1 Phase 1 — Implemented Architecture

The Phase 1 system is implemented as a modular, locally hosted voice pipeline. The complete call path is:

MicroSIP Softphone (Caller)
↓ Asterisk 22.10.1 (SIP Call Handling)
↓ AudioSocket (Audio Transport)
↓ Python Audio Gateway
↓ WebRTC VAD (Speech Detection)
↓ Faster-Whisper STT (small, CUDA, float16)
↓ RAG Service
↓ Sentence Transformers (Query Embedding)
↓ ChromaDB (Vector Retrieval)
↓ Ollama + Qwen3:4B (Response Generation)
↓ Piper TTS — en_US-lessac-medium
↓ 8 kHz PCM Audio
↓ AudioSocket
↓ Asterisk
↓ MicroSIP Softphone (Caller)

4.2 Component Descriptions

Component	Technology (Phase 1)	Role
Telephony	Asterisk 22.10.1 + SIP + MicroSIP	SIP-based call handling; routes the inbound SIP call to the AudioSocket channel
Audio Transport	Asterisk AudioSocket	Transfers raw PCM audio bidirectionally between Asterisk and the Python gateway
Voice Activity Detection	WebRTC VAD	Segments incoming audio to detect speech boundaries
Speech-to-Text	Faster-Whisper (small, CUDA, float16)	Transcribes caller speech to text locally with GPU acceleration
Content Ingestion	Python, Requests, BeautifulSoup, PyMuPDF, Tesseract OCR	Extracts and cleans institutional content from web pages and PDFs
Embeddings	Sentence Transformers	Converts knowledge chunks and queries into dense vector representations

Vector Database	ChromaDB	Stores and retrieves embedded institutional knowledge chunks
RAG Pipeline	Custom Python retrieval and reranking	Retrieves relevant context chunks; applies relevance-based reranking; attaches metadata
LLM	Ollama + Qwen3:4B (locally hosted)	Generates grounded natural-language responses from retrieved context
Text-to-Speech	Piper TTS — en_US-lessac-medium	Synthesises the text response into 8 kHz PCM speech audio
Backend Orchestration	Python	Coordinates the full audio → STT → RAG → LLM → TTS → audio pipeline

4.3 Planned Phase 2 Escalation Logic (Not Yet Implemented)

Phase 2 will introduce agentic call routing and escalation. The agent will transfer a call to the human receptionist when any of the following conditions are met:

- The RAG retrieval returns no sufficiently relevant context and the query does not fall into an authenticated student-data category.
- The caller explicitly requests to speak to a human.
- The query involves administrative decisions, complaints, or matters requiring institutional approval.
- The Phase 2 caller authentication step fails after the configured number of retries.

On escalation, the system will: (a) inform the caller of the transfer; (b) generate a brief text summary of the conversation; (c) route the call to the reception desk via the telephony system; and (d) deliver the summary to the receptionist prior to the call connecting. This behaviour is planned and not part of the current Phase 1 implementation.

5. Knowledge Base — Data Sources and Ingestion

5.1 Phase 1 Knowledge Sources

The Phase 1 knowledge base is populated exclusively from publicly available Rajalakshmi Engineering College information. The indexed sources are:

Website Content

- Institutional pages covering: About, Academics, Admissions, Departments, Faculty, Placements, Research, Facilities, Events, and department-specific sub-pages.

PDF Documents

- Certificate for CGPA to Percentage Conversion
- Certificate for Medium of Instruction
- R2023 CSE Curriculum and Syllabus
- REC 2023 UG Regulations

Scanned certificate PDFs were processed with Tesseract OCR where standard text extraction was not applicable. The Phase 1 knowledge base contains a total of 767 indexed chunks.

5.2 Ingestion Pipeline

The implemented ingestion pipeline follows these stages:

7. Website content is collected using Python with Requests and BeautifulSoup from publicly available institutional pages.
8. PDF documents are parsed using PyMuPDF. Scanned or image-based PDFs are processed through Tesseract OCR to extract text.
9. Extracted text is cleaned to remove navigation artefacts, repeated headers, and non-content elements.
10. Cleaned text is chunked with overlap to preserve contextual continuity across chunk boundaries.
11. Chunks are embedded using Sentence Transformers and stored in ChromaDB with source, document title, and page metadata for attribution in responses.

A re-ingestion process can be run when the institutional knowledge base requires updating to reflect new notices, revised regulations, or updated academic information.

6. Technology Stack

6.1 Phase 1 — Implemented Stack

Layer	Technology	Rationale
Telephony	Asterisk 22.10.1 + SIP + MicroSIP softphone	Open-source PBX providing a complete SIP-based local telephony environment for prototype development
Audio Transport	Asterisk AudioSocket	Native Asterisk protocol for raw PCM audio streaming to an external application over TCP
Voice Activity Detection	WebRTC VAD	Lightweight, real-time speech segment detection; avoids sending silence to the STT engine
Speech-to-Text	Faster-Whisper (small, CUDA, float16)	Local GPU-accelerated STT with strong English-language accuracy; avoids cloud API dependency
Web Scraping	Python + Requests + BeautifulSoup	Reliable extraction of institutional website content from static HTML pages
PDF Extraction	PyMuPDF + Tesseract OCR	Handles both text-native and scanned image-based PDF documents
Embeddings	Sentence Transformers	Open-source, locally hosted embedding model; no per-call embedding API cost

Vector Database	ChromaDB	Lightweight local vector store appropriate for single-institution corpus scale
LLM	Ollama + Qwen3:4B (locally hosted)	Fully local inference; no cloud API dependency; suitable for institutional data privacy in a prototype
Text-to-Speech	Piper TTS — en_US-lessac-medium	Fast, locally hosted neural TTS; produces natural speech output without external API calls
Backend	Python	Coordinates all pipeline components: audio gateway, VAD, STT, RAG, LLM, and TTS

6.2 Phase 2 — Planned Additions (Not Yet Implemented)

Layer	Planned Technology	Purpose
Agentic Orchestration	LLM native function/tool calling or LangGraph	Enables the LLM to route between RAG retrieval and DigiCampus API tools dynamically
Caller Authentication	Student ID + PIN (DTMF) or OTP via SMS	Voice-channel identity verification before any personalised data is retrieved
DigiCampus Integration	Read-only REST API calls to DigiCampus endpoints	Live retrieval of attendance, marks, fee status, and placement eligibility per authenticated student
Call Transfer	Asterisk call transfer with pre-transfer transcript summary	Escalation to human receptionist with conversation context delivered before call connects
Audit Logging	Structured call logs in a local database	Enables accuracy monitoring, audit, and iterative improvement of the knowledge base

7. Phase 1 — Evaluation Results

7.1 Retrieval and Answer Accuracy

A Phase 1 evaluation was conducted using a set of 20 test queries. The query set included both normal institutional information queries and negative (out-of-scope) queries designed to test the agent's refusal behaviour.

Metric	Result
Total evaluation queries	20
Normal information queries	18
Negative / out-of-scope queries	2
Retrieval correct (Hit@5)	18 / 18 — 100%
Answer accuracy on normal queries	18 / 18 — 100% on evaluated test set

Negative query handling (refusal accuracy)	2 / 2 — 100% on evaluated test set

100% retrieval and answer accuracy was achieved across the 20-query evaluation set. These results reflect performance on the defined test set and should be interpreted accordingly; performance may vary across query types and phrasing not covered by the evaluation set. The evaluation confirms that the RAG pipeline correctly grounds responses in the indexed institutional knowledge and reliably declines out-of-scope queries rather than speculating.

7.2 Latency Evaluation

A separate Phase 1 latency evaluation measured the AI processing latency for the voice pipeline across each component. The measurements represent the average AI processing time from STT input through TTS audio preparation and do not include telephony signalling or network transmission delays.

Pipeline Stage	Measured Average Latency
Speech-to-Text (Faster-Whisper)	0.25 seconds
RAG Retrieval	0.07 seconds
LLM Generation (Ollama + Qwen3:4B)	18.55 seconds
Text-to-Speech (Piper TTS)	2.46 seconds
Total AI Processing Latency	21.35 seconds

The Phase 1 local evaluation recorded an average total AI processing latency of 21.35 seconds. LLM generation is the primary computational bottleneck, contributing approximately 86.9% of the measured pipeline latency. STT and RAG retrieval components are individually fast (under 0.3 seconds combined), and TTS synthesis contributes approximately 2.46 seconds.

This latency is attributable to local CPU/GPU inference constraints in the prototype environment. Reducing end-to-end latency toward a conversationally responsive range is identified as a primary objective for Phase 2, with potential approaches including cloud GPU deployment, model quantisation optimisation, and streaming TTS output before full LLM generation is complete.

8. Advantages and Limitations

8.1 Advantages

- Automated handling of routine institutional queries reduces dependency on manual reception staff for repeatable inquiries.
- Knowledge-grounded responses: the agent answers only from indexed institutional content, providing consistent information regardless of which operator would otherwise have taken the call.
- Fully local architecture in Phase 1: no institutional data is transmitted to external cloud services during prototype operation.

- Modular pipeline design allows individual components (STT, LLM, TTS, retrieval) to be upgraded or replaced independently.
- Measurable evaluation: retrieval accuracy, answer accuracy, refusal behaviour, and latency have been quantified and documented.
- Extensible toward Phase 2 personalised student services and authenticated ERP integration without redesigning the core pipeline.

8.2 Limitations and Constraints

- Local hardware dependency: Phase 1 inference performance is bounded by the available local GPU. LLM generation latency (18.55 seconds average) is the primary bottleneck and is not suitable for conversationally responsive deployment on the current hardware configuration.
- STT performance is sensitive to speech clarity, background noise, and accent variation. Performance on Tamil-English code-mixed speech or low-quality telephony audio may differ from the evaluated conditions.
- The knowledge base is limited to the specific institutional sources ingested in Phase 1. Queries about topics not covered by the indexed content will result in a polite decline rather than an accurate answer, which requires ongoing knowledge base maintenance.
- Phase 1 does not access personalised student information. Queries about individual attendance, results, or fee status cannot be answered until Phase 2 is implemented.
- Phase 2 is contingent on institutional IT approval for DigiCampus API access, which is outside the direct control of the development team.

9. Phase Status and Milestones

Phase 1 implementation and evaluation have been completed. Mentor review and approval constitute the final transition milestone before Phase 2 planning begins.

Milestone	Description	Status
M1	College website and selected PDF ingestion pipeline completed; knowledge base of 767 chunks populated and searchable in ChromaDB.	COMPLETED
M2	RAG backend integrated with locally hosted Qwen3:4B LLM via Ollama; evaluated using text-based test queries.	COMPLETED
M3	STT → RAG → LLM → TTS voice pipeline integrated end-to-end with Python audio gateway.	COMPLETED
M4	Asterisk/SIP/MicroSIP telephony integration with AudioSocket completed; end-to-end call flow operational.	COMPLETED
M5	20-query retrieval, answer accuracy, and refusal evaluation completed; latency evaluation completed; CSV results and graphs generated.	COMPLETED

M6	Mentor review of Phase 1 results; Phase 1 approval; decision on Phase 2 scope and DigiCampus API access.	PENDING
M7	Caller authentication flow (Student ID + PIN/OTP) design and implementation.	PLANNED
M8	DigiCampus API integration or sandboxed equivalent: attendance, marks, fee, placement eligibility.	PLANNED
M9	Escalation-to-human-receptionist flow: call transfer and transcript summary delivery.	PLANNED
M10	End-to-end Phase 2 evaluation; final documentation and demonstration.	PLANNED

10. Open Questions for Mentor Discussion

12. DigiCampus API access: Does the college IT department expose a documented REST API for student data, or would Phase 2 require an alternative approach such as a read-only database connector or an intermediary microservice? This is the single largest external dependency for Phase 2 and must be clarified before development begins.
13. Authentication mechanism: What caller identity verification mechanism is institutionally acceptable for a voice channel (Student ID + DTMF PIN, OTP via SMS, or another approach)?
14. Human receptionist transfer endpoint: Does the college telephony infrastructure support SIP-based call transfer to a defined reception extension, and what is the appropriate escalation endpoint for Phase 2?
15. Data retention and privacy: Should call transcripts and anonymised query logs be stored for evaluation and quality-improvement purposes, and if so, what is the permissible retention period and storage location under the college's IT policy?
16. Phase 2 feature priority: Of the planned capabilities (authenticated student queries, personalised data retrieval, human escalation), which should be prioritised first given API access constraints and development time?
17. Latency optimisation approach: Should Phase 2 target cloud GPU deployment for LLM inference to reduce end-to-end latency, or should the system remain locally hosted with model-level optimisation (quantisation, smaller model variants)?
18. Phase 2 evaluation criteria: What metrics and acceptance thresholds should be used to evaluate Phase 2 (e.g., authentication success rate, DigiCampus query accuracy, escalation correctness), and should this mirror the Phase 1 evaluation methodology?

11. Academic and Career Value

This project spans multiple technical disciplines that are rarely combined in a single undergraduate project. The table below distinguishes skills demonstrated through Phase 1 implementation from those planned in Phase 2.

Skill Domain	Phase 1 — Demonstrated	Phase 2 — Planned

RAG Engineering	Chunking, Sentence Transformer embeddings, ChromaDB retrieval, relevance reranking, source metadata, grounding constraints	—
Document Ingestion	Web scraping (BeautifulSoup), PDF parsing (PyMuPDF), OCR (Tesseract), content cleaning	—
Speech Pipeline	Real-time STT (Faster-Whisper), TTS (Piper), WebRTC VAD, 8 kHz PCM audio handling	—
Telephony	Asterisk 22.10.1, SIP, MicroSIP, AudioSocket, audio gateway implementation	Call transfer, escalation logic
LLM Integration	Locally hosted LLM via Ollama (Qwen3:4B), prompt grounding, refusal handling	Tool calling, agentic routing
Evaluation Methodology	Retrieval accuracy, answer accuracy, refusal accuracy, latency measurement per component	Phase 2 evaluation design
Security and Privacy	Knowledge base restricted to public data; no student PII processed	Authentication, per-user data isolation, audit logging
Agentic AI	—	Tool-use orchestration, DigiCampus API integration, confidence-based routing

The combination of real telephony integration (Asterisk/SIP), a fully local AI stack (STT, retrieval, LLM, TTS), and a quantified evaluation methodology — applied to a genuine institutional problem — provides a technically credible and independently presentable project at the undergraduate final-year level.

12. Conclusion

Phase 1 of the Voice-Based AI Receptionist Agent has been implemented and evaluated. The system establishes a complete, locally hosted voice pipeline capable of receiving SIP calls, understanding natural-language queries, retrieving relevant institutional knowledge, generating grounded responses, and delivering them as synthesised speech — all without accessing personal student data.

The Phase 1 evaluation on a 20-query test set recorded 100% retrieval accuracy (Hit@5), 100% answer accuracy, and 100% refusal accuracy on out-of-scope queries. The measured average AI processing latency is 21.35 seconds, with LLM generation identified as the primary bottleneck. Latency reduction through cloud GPU deployment or model optimisation is a defined objective for Phase 2.

Phase 2 — planned following mentor review and institutional approval — will introduce authenticated student data retrieval via DigiCampus API integration, agentic tool-calling, and intelligent call escalation to a human receptionist. Phase 1 approval and a decision on Phase 2 scope remain as the immediate next steps.